

CoderCommands MCP System

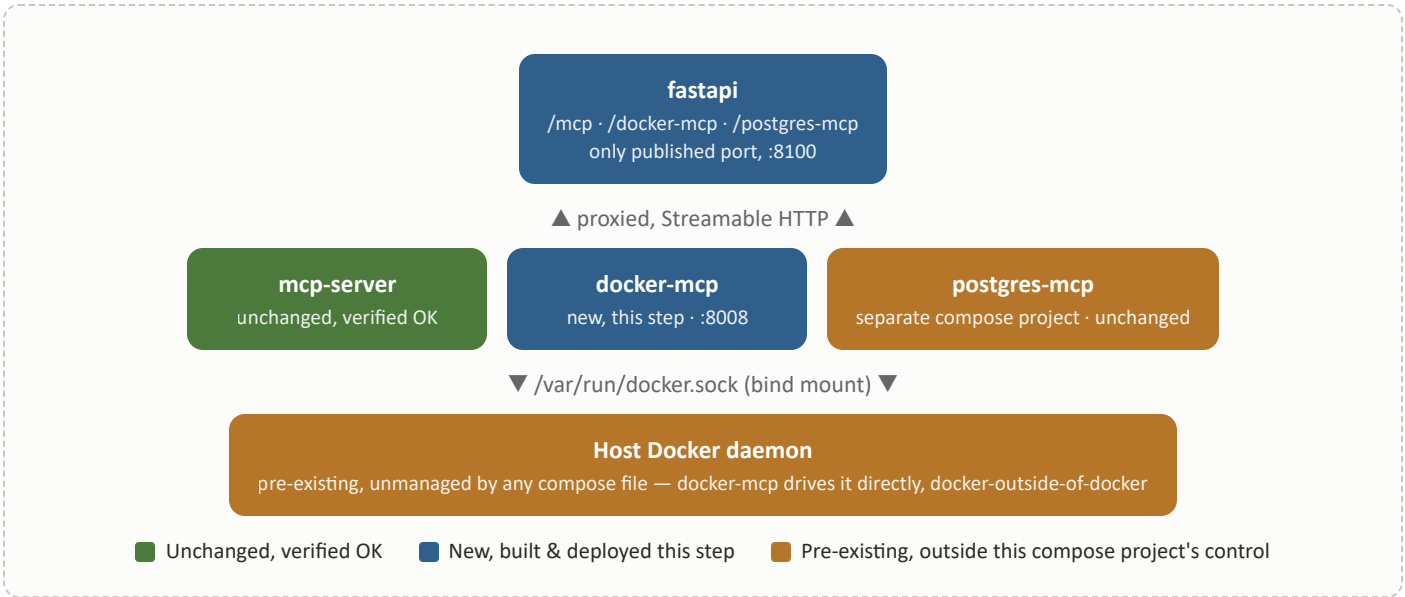
Step 13 Record — Docker MCP Containerization & Gateway Deployment

DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	13 — Docker MCP Containerization & Gateway Deployment (post-plan addendum)
DATE	2026-08-15
REVISION	1.0
STATUS	Complete — built, deployed, verified end-to-end, registered with Claude Code

1. Purpose & Scope

Bring a third MCP server, `docker-mcp` (a fork of QuantGeekDev/docker-mcp living outside this repo at `backend/docker-mcp/docker-mcp`), into the same topology already proven for `mcp-server` (Step 6/7) and `postgres-mcp` (Step 11/12): its own container, reachable only from `fastapi`, reverse-proxied out through one published port. Unlike `postgres-mcp`, this one lives entirely inside the ManPage compose project — no second compose project, no shared external network. Unlike `mcp-server`, its job is to let an MCP client drive the *host's* Docker daemon (create containers, deploy compose stacks, read logs), which is the source of every containerization decision below.

2. Final Verified Topology



`docker-mcp` joins only `codercommands-net` (no `mcp-shared`, no host port) — the network boundary that keeps a server with effectively root-equivalent host control from being reachable by anything other than `fastapi`. See §4.3.

3. Deployment Step Log

Every action taken, in order. No incidents required backtracking this time — the DNS-rebinding and Docker-CLI issues in §4 were both anticipated from precedent (Step 6's `mcp-server` fix, and reading `docker_executor.py` before writing the Dockerfile) and designed around up front, rather than hit and fixed after a failed request.

#	Action	Command / Change	Outcome
1	Locate the source	Found via <code>docker inspect codercommands-fastapi</code> → compose project path, then <code>backend/docker-mcp/docker-mcp</code>	OK
2	Inspect the fork	<code>git log / git status / git diff</code> — confirmed origin is QuantGeekDev/docker-mcp, with an uncommitted <code>streamable-http</code> transport already added to the working tree	Found existing state
3	Read Docker dependency	<code>docker_executor.py / handlers.py</code> — shells out to the <code>docker</code> CLI directly (<code>shutil.which</code>) and via <code>python_on_whales.DockerClient()</code> , which also drives the CLI, not a Python Docker SDK	Design input
4	Close a DNS-rebinding gap	Edited <code>server.py</code> : pass <code>security_settings</code> (allow-list <code>docker-mcp:8008</code>) to <code>StreamableHTTPSessionManager</code> — see §4.1	OK
5	Scope the build context	Added <code>.dockerignore</code> (<code>.venv</code> , <code>.git</code> , <code>__pycache__</code>)	OK
6	Write the Dockerfile	Multi-stage: <code>docker</code> CLI binary copied from <code>docker:27-cli</code> , then <code>pip install .</code> on <code>python:3.12-slim</code> — see §4.2	OK
7	Add the compose service	<code>docker-compose.yml</code> : new <code>docker-mcp</code> service, <code>/var/run/docker.sock</code> bind mount, <code>codercommands</code> network only, no published port	OK
8	Wire the gateway env	<code>fastapi</code> service: added <code>DOCKER_MCP_UPSTREAM_URL=http://docker-mcp:8008</code> , added to <code>depends_on</code>	OK
9	Add the proxy route	<code>api/main.py</code> : <code>_docker_client + /docker-mcp{path}</code> route, mirroring <code>proxy_mcp</code> (Streamable HTTP, no <code>/messages</code> side-channel needed)	OK
10	Build the image	<code>docker compose build docker-mcp</code>	OK — 35s
11	Start it	<code>docker compose up -d docker-mcp</code>	OK
12	Rebuild & restart the gateway	<code>docker compose build fastapi && docker compose up -d fastapi</code>	OK, healthy
13	Verify Docker CLI reaches the host daemon	<code>docker exec codercommands-docker-mcp docker ps</code>	OK — listed host containers
14	MCP handshake through the gateway	<code>curl -X POST :8100/docker-mcp</code> — <code>initialize</code>	OK — 200, session issued
15	Confirm tools are exposed	<code>tools/list</code> via the same session	OK — 4 tools
16	Register with Claude Code	<code>claude mcp add --transport http docker http://127.0.0.1:8100/docker-mcp</code>	OK — Connected
17	Update docs	<code>README.md</code> : topology diagram, service table, registration command, limitations note	OK

4. Challenges & How They Were Resolved

4.1 The MCP SDK's DNS-Rebinding Default Would Have Blocked Every Proxied Request

`StreamableHTTPSessionManager` defaults to `enable_dns_rebinding_protection=True` with `allowed_hosts=[]` — an *empty* allow-list, which rejects every `Host` header, not just untrusted ones. A request arriving through the gateway carries `Host: docker-mcp:8008` (the real internal service name `httpx` fills in), which would 400 against the SDK's own default. This is exactly the class of problem `mcp-server` already solved in Step 6 — recognized from that precedent before ever starting the container, so it never surfaced as a live failure the way the `postgres-mcp` SSE/prefix issue did in Step 12. Fixed by passing an explicit `TransportSecuritySettings` allow-listing `docker-mcp:8008 / localhost:8008 / 127.0.0.1:8008`, keeping the protection on rather than disabling it — correct because this container has no published host port, so "reachable at all" already means "reachable from a trusted caller."

4.2 Getting the `docker` CLI Into the Image Without Full Docker-in-Docker

`docker_executor.py` and `python-on-whales` both shell out to a `docker` binary on `PATH` — this server was never going to run its own daemon, it needed to control the *host's* daemon (`docker-outside-of-docker`), which only needs the CLI plus a socket, not a nested `dockerd`. The two ways to get the CLI onto a slim Python base image are apt-installing Docker's own repo (a GPG key, a repo file, a larger image) or lifting the single static binary out of the official CLI image. Used a multi-stage build — `FROM docker:27-cli AS docker-cli`, then `COPY --from=docker-cli /usr/local/bin/docker /usr/local/bin/docker` — for one binary instead of a whole package repository.

4.3 Security Tradeoff: the Host Docker Socket Is Not a Sandboxed Capability

Bind-mounting `/var/run/docker.sock` gives this container effectively root-equivalent control of the host: it can create a container with an arbitrary bind mount (including the host filesystem), read any other container's logs, or stop anything running. This is inherent to what the tool does, not a bug to fix — flagged here rather than glossed over, because the only thing making it safe to run at all is that `docker-mcp` joins *only* `codercommands-net`: no published host port, no `mcp-shared` join, reachable exclusively from `fastapi` by service name. That network boundary is now the actual security boundary for this service and should be treated as load-bearing before anyone widens it (e.g. adding a host port for direct debugging).

4.4 Inherited, Not Authored: the Streamable-HTTP Transport Itself

Worth recording precisely: the fork's `--transport streamable-http` flag, its argparse wiring in `__init__.py`, and `create_streamable_http_app()` in `server.py` were already present as uncommitted changes in the fork's working tree before this step began (confirmed via `git diff` against `origin/main`) — upstream `QuantGeekDev/docker-mcp` only ships `stdio`. This step's contribution on top of that existing code is the DNS-rebinding fix (§4.1) and everything needed to actually run it as a container behind the gateway (§4.2 onward); it did not add the transport from scratch.

5. File / Component Register (Final State)

Ref	File / Component	Final State
R1	backend/docker-mcp/docker-mcp/Dockerfile	New. Multi-stage: <code>docker</code> CLI copied from <code>docker:27-cli</code> , then <code>pip install . on python:3.12-slim</code> . CMD runs <code>docker-mcp --transport streamable-http --host 0.0.0.0 --port 8008</code> .
R2	backend/docker-mcp/docker-mcp/.dockerignore	New. Excludes <code>.venv</code> , <code>.git</code> , <code>__pycache__</code> from the build context.
R3	backend/docker-mcp/docker-mcp/src/docker_mcp/server.py	Edited (on top of pre-existing uncommitted changes, §4.4). <code>create_streamable_http_app()</code> now passes an explicit <code>TransportSecuritySettings</code> allow-listing the internal service host — see §4.1.
R4	docker-compose.yml (ManPage)	New <code>docker-mcp</code> service: <code>build: ../backend/docker-mcp/docker-mcp</code> , <code>/var/run/docker.sock</code> bind mount, <code>codерcommands</code> network only, no published port, <code>restart: unless-stopped</code> . <code>fastapi</code> gained <code>DOCKER_MCP_UPSTREAM_URL</code> and a <code>depends_on</code> entry.
R5	api/main.py	<code>_docker_client</code> <code>httpx</code> client (lifespan-managed) and a new <code>/docker-mcp{path}</code> proxy route, structurally identical to the existing <code>/mcp</code> route. Module docstring updated to mention it.
R6	README.md	Architecture diagram, service table, fork provenance note, <code>claude mcp add</code> line, and a new limitations bullet on the socket's blast radius.
R7	Host Docker daemon	Untouched; now additionally reachable, by design, from inside the <code>docker-mcp</code> container via the bind-mounted socket.

6. Verification Record

Check	Method	Result
Image builds cleanly	<code>docker compose build docker-mcp</code>	PASS
Container starts and stays up	<code>docker compose ps</code> — Up, no restart loop	PASS
<code>docker</code> CLI in the container reaches the <i>host</i> daemon	<code>docker exec codерcommands-docker-mcp docker ps</code> — listed the same containers as the host, including itself	PASS
Gateway proxies the MCP handshake	<code>POST :8100/docker-mcp initialize</code> — 200, <code>Mcp-Session-Id</code> issued, no DNS-rebinding rejection	PASS
Tools are actually exposed	<code>tools/list</code> in the same session — returned <code>create-container</code> , <code>deploy-compose</code> , <code>get-logs</code> , <code>list-containers</code>	PASS
Existing routes have no regression	<code>GET :8100/healthz</code> — <code>{"status": "ok"}</code>	PASS
<code>claude mcp add</code> registration	<code>claude mcp add --transport http docker ...</code> , then <code>claude mcp get docker</code>	PASS — Connected

7. Next Step / Open Dependencies

Registered. `docker` is now in this project's local Claude Code config alongside `codерcommands` and `postgres`; a fresh session picks up all three automatically. Every check in §6 passes — nothing is pending from this step.

Open item for whoever picks this up next: `docker-mcp`'s four tools are currently unauthenticated and unaudited at the MCP layer — any client that can reach `/docker-mcp` can create, stop, or read logs from containers on this host with no per-tool confirmation. That's acceptable today only because the network boundary in §4.3 is the sole gate. If this ever needs to be reachable more broadly than "this one gateway container," add real authorization before widening it, not after.